Adaptive Task Allocation for Heterogeneous Bio-Inspired Agents

Detailed Research Plan

Final Year B.Tech Project, 2026–27
Department of Computer Science and Engineering
Delhi Technological University

Submitted to Dr. Yeshwant Singh, Group Faculty Coordinator (Group A4), on 12 October 2026

1 Group Details

Project title	Adaptive Task Allocation for Heterogeneous Bio-Inspired Agents
Group ID	Group Key A4, BTP Group No. GP105
Group Faculty Coordinator	Dr. Yeshwant Singh
Guide	Prof. Shailender Kumar, Department of CSE
Co-guide	None

Name	Roll No.	Programme and year	Signature
Nikshay Yadav	23/CS/284	B.Tech CSE, final year	
Madan Raj Murugesan	23/CS/234	B.Tech CSE, final year	
Aryan Varshney	_____	B.Tech CSE, third year (junior member)	

Signature of the guide: _____

2 Abstract and Problem Statement

2.1 Abstract

Multi-agent systems built on large language models (LLMs) usually fix one way of working together before the task is seen: a debate, a pipeline of roles, or a manager with workers. Real tasks are mixed. One part needs a search over many candidate paths, another needs several independent opinions to be reconciled, and a third is routine work that one worker can finish alone. When the same coordination pattern is applied to all of them, the system pays for messages and tokens that do not improve the answer, and recent studies report that such systems often fail to beat a well-prompted single agent. This project asks whether a coordinator that picks a different coordination mechanism for each subtask can do better within the same budget. The mechanisms are taken from animal behavior and written as ordinary algorithms: reinforced search (ant), evidence accumulation with cross inhibition (bee), bounded local execution (octopus), tool-prerequisite planning (crow) and checkpoint-based recovery (starfish), with two more kept as extensions. We will build the framework, profile each mechanism on planning, tool-use and reasoning tasks whose answers can be checked automatically, and learn a lightweight selector from the recorded outcomes. Adaptive allocation will be compared with a single agent, the best single mechanism, fixed and random allocation, and a conventional coordinator with specialist agents that is given the same model, tools and budget. The result will be a working prototype and a measured account of when a mix of mechanisms helps and when it does not.

2.2 Problem statement

An LLM multi-agent system splits a task among agents that exchange messages. The split is useful only if the way the agents coordinate suits the work. We expect a subtask with many possible solution paths to benefit from exploration that remembers what worked, a subtask with a few competing answers to need those answers compared and one of them chosen, and a well-specified subtask to need no discussion at all, since involving the coordinator at every step only adds cost. These are hypotheses, and RQ1 tests them. Current systems mostly choose one pattern for the whole task. The evidence reviewed in Section 3 shows the consequence: the same architecture that improves one kind of task by a large margin makes another kind worse [6], and many failures can be traced to system design and coordination and not only to the underlying model [3].

We state the problem as follows. A task is decomposed into a dependency graph $G = (V, E)$ of subtasks. A pool $\mathcal{M} = \{m_1, \ldots, m_K\}$ of coordination mechanisms is available behind one interface, together with a direct route that answers a subtask with a single model call. For each subtask v that becomes ready, the coordinator chooses $\pi(v) \in \mathcal{M} \cup \{\text{direct}\}$ from features it can observe before execution and from the outcomes it has recorded so far. It never sees the reference answer of a task, only the runtime checks described in Section 4.4. The aim is to maximize the probability that the final answer is correct while the total cost, counted in tokens and model calls and including the coordinator's own share, stays within a budget B. A wrong choice is paid for twice, once in wasted budget and once in the reallocation that follows, so routing errors and communication overhead are part of the problem and not side effects.

The project sets out to measure three things: whether mechanisms that share one backbone model differ in what they are good at; if they do, whether a selector can exploit the difference using cheap features; and whether the biological mechanisms add anything over well-designed conventional agents once compute is held equal.

2.3 Objectives and research questions

O1. Build a modular coordinator with a common interface for heterogeneous mechanisms, starting with ant, bee and octopus and designed to take all seven proposed modules.

O2. Measure the strengths of each mechanism across task families and build capability profiles from correctness, cost and latency.

O3. Design allocation policies, first rule-based and then learned from recorded outcomes, and test them within a fixed budget.

O4. Separate the effect of the mechanisms from the effect of extra computation, using matched baselines and component ablations.

O5. Test recovery from injected failures and find the point at which coordination stops paying for itself.

The objectives correspond to the five research questions of the approved synopsis. **RQ1:** Do the mechanisms have complementary strengths? **RQ2:** Can dynamic allocation improve performance within a fixed budget? **RQ3:** What value do the biological mechanisms add? **RQ4:** Can the framework adapt to failures and changing task requirements? **RQ5:** When does coordination become inefficient?

3 Brief Literature Review

Fixed workflows and their cost. Most LLM multi-agent frameworks fix the roles and the communication pattern when the system is designed. He, Treude and Lo [4] review 71 primary studies of such systems in software engineering and place the optimization of agent collaboration as the second phase of their research agenda, after the capabilities of individual agents. The cost of fixed designs is now documented. Cemri et al. [3] built a taxonomy of 14 failure modes in three groups (system design, inter-agent misalignment and task verification) from expert analysis of 150 execution traces, and used it to

annotate more than 1,600 traces from seven popular frameworks. Wang et al. [7] found that a single agent with a strong prompt comes close to the best multi-agent discussion method on a range of reasoning benchmarks. Zhang et al. [8] treated redundant communication as a problem in its own right and showed that many messages can be pruned. In a controlled study, Kim et al. [6] compared a single agent with four multi-agent architectures on six benchmarks and three model families under standardized tools, prompts and compute. The relative change in performance against the single agent ranged from +80.8% on a decomposable financial reasoning task to −70.0% on sequential planning. Their conclusion, that coordination helps only when it matches the structure of the task, is the starting point of this project.

Adaptive topology and routing. A second line of work lets the structure depend on the task. GPTSwarm [2] describes agents as computational graphs and optimizes both the prompts at the nodes and the connections between them. DyLAN [9] selects a team for a task using an agent importance score. G-Designer [10] generates a communication topology for each task with a variational graph auto-encoder, and AFlow [11] searches over workflows written as code using Monte Carlo tree search. RouteLLM [12] learns to send each query to a stronger or a weaker model. MasRouter [1] brings these decisions together: a cascaded controller chooses the collaboration mode, the agent roles and the LLM for each role, and reports up to 52% lower overhead than earlier methods on HumanEval. Work from 2026 routes at a finer grain. GraphPlanner [13] chooses a model and an agent role at every step of a workflow, using a graph memory of past interactions. AdaptOrch [14] picks one of four topologies from the dependency graph of the task. Symphony-Coord [15] is the closest to our coordinator. It routes each subtask to an executor agent with a LinUCB bandit over task and agent features, updates the bandit from execution feedback, and reports recovery under agent failures and distribution shift. Adaptive routing of subtasks with feedback is therefore established. In all of these systems the candidates being selected are agents, roles, models or graph shapes.

Bio-inspired coordination with LLMs. Wu et al. [5] survey the two-way relationship between LLMs and evolutionary computation, including LLMs that generate and improve optimization algorithms. Two recent preprints apply swarm ideas to agent coordination directly, and both draw on ant colonies. AMRO-S [16] models routing as the selection of a path through layers of agents and applies ant colony optimization, with separate pheromone memories for different task types and a small fine-tuned model that infers the intent of a query; its ablations concern the router. SwarmSys [17] cycles explorer, worker and validator agents and uses a pheromone-inspired reinforcement rule for task allocation; its ablations remove the matching and the roles. There is also a skeptical reading. Rahman et al. [18] re-implemented Boids and ant colony optimization with LLM agents and found that the LLM version of Boids needed about 300 times the computation time of the classical one. Outside the LLM setting, Damaševičius and Maskeliūnas [19] combine the bat and krill herd algorithms by switching agents between them according to performance, an early form of the selection studied here. Camacho-Villalón et al. [20] analyzed six highly cited animal-named optimizers component by component and argued that their metaphors hide ideas that already existed under other names. We take this as a design rule: every module in this project is specified by what it computes, and each is tested against plain counterparts that lack its distinguishing components.

Biological basis and software counterparts. Seeley et al. [21] showed that scout bees committed to one nest site deliver stop signals to scouts advertising a rival site, and that this cross inhibition lets a swarm settle on one option even when two are equally good. Its nearest software relative is majority voting over independent samples [22], which has no inhibition and no quorum. Sumbre et al. [23] showed that the arm extension of the octopus is produced by a motor program in the arm itself, with the brain issuing the command but not the details, a division that resembles a coordinator handing a bounded job to a worker. Taylor et al. [24] observed New Caledonian crows using a short tool to obtain a longer one that could reach food. In LLM systems the corresponding ideas are tool making and reuse [25], planning of function calls as a dependency graph [26], and tool learning more generally [27]. Kano et al. [28] built a brittle-star-like robot that adapts immediately to the loss of arms through decentralized control. For LLM agents, Huang et al. [29] injected agents that make mistakes into six systems and found that

hierarchical structures lost the least performance (5.5%), while Zhang et al. [30] showed that even the best automatic method names the agent responsible for a failure only 53.5% of the time. Theraulaz et al. [31] modeled division of labor in insect colonies by letting each completed task lower an individual's response threshold for that task, so that specialists emerge without central assignment; a selector that reinforces the choices that worked is a loose analogue. Experience stores for LLM agents [32, 33] are the related software mechanism behind the proposed elephant module.

What this project adds. Adaptive routing, selection from feedback and recovery all exist in the work above, and we do not claim to introduce them. The open question is narrower. Table 1 lists what the closest systems select: agents, roles, models and topologies. We are not aware of a study that implements several coordination mechanisms with different control flow, memory and stopping rules behind one interface, runs them on the same subtasks at the same budget, and reports which mechanism suits which kind of subtask and which component of each is responsible. This project is an empirical study of that question. Its selector deliberately reuses a known method, the LinUCB routing of [15], so that a difference in results comes from what is selected and not from how the selection is made.

Table 1. Closest prior systems and the position of this project.

System	Candidates selected	Selection unit	Method
GPTSwarm [2]	Node prompts and graph edges	Task family	Graph optimizers
AFlow [11]	Whole workflow, as code	Task family	Monte Carlo tree search
G-Designer [10]	Communication topology	Query	Graph auto-encoder
MasRouter [1]	Collaboration mode, roles, LLMs	Query	Cascaded controller network
AMRO-S [16]	Path through layers of agents	Query	Ant colony optimization
AdaptOrch [14]	One of four topologies	Task	Routing on the dependency graph
GraphPlanner [13]	LLM and agent role	Workflow step	Reinforcement learning
SwarmSys [17]	Agent for an event	Sub-event	Pheromone-style reinforcement
Symphony-Coord [15]	Executor agent	Subtask	LinUCB bandit
This project	Coordination mechanism	Subtask	Rules, then a LinUCB bandit

3.1 Five primary references

References [1]–[5] are the five primary references; all appeared in 2024 or 2025. Table 2 summarizes how each is used. The older biological papers in the list are cited as sources of the mechanisms and not as recent related work.

4 Proposed Methodology

4.1 Architecture

Figure 1 shows the planned system. The coordinator decomposes the incoming task into a dependency graph, selects an executor for each subtask that is ready, dispatches it with a share of the budget and applies the runtime checks of Section 4.4 to the result. A result that passes is aggregated into the answer. One that fails is reallocated, which is a new selection made with the failed attempt on record. Every executor implements the same call: it takes a subtask, its context and a budget, and returns the output, a confidence estimate, the cost (prompt and completion tokens, model calls and elapsed time), a status flag and the full trace.

The decomposer, the runtime checks, the tools, the backbone model and the prompt templates for shared

Table 2. Primary references and their use in the project.

Ref.	Venue and year	Use in this project
[1]	ACL, 2025	Formulation of routing in multi-agent systems; source of the reasoning sets we reuse.
[2]	ICML, 2024	View of an agent system as a graph whose connections can be optimized.
[3]	NeurIPS, 2025	Failure taxonomy, used to label a sample of our failed runs.
[4]	ACM Trans. Softw. Eng. Methodol., 2025	Review of role-based multi-agent systems; reference for the roles in the conventional specialist baseline.
[5]	IEEE Trans. Evol. Comput., 2025	Survey connecting LLMs with evolutionary and swarm computation.

steps are common to every system we compare, including the baselines. Executors differ in control flow, communication rules, memory updates and stopping conditions. Sharing the rest removes the obvious confounds, and the ablations of Section 4.4 are what tie an effect to a particular component.

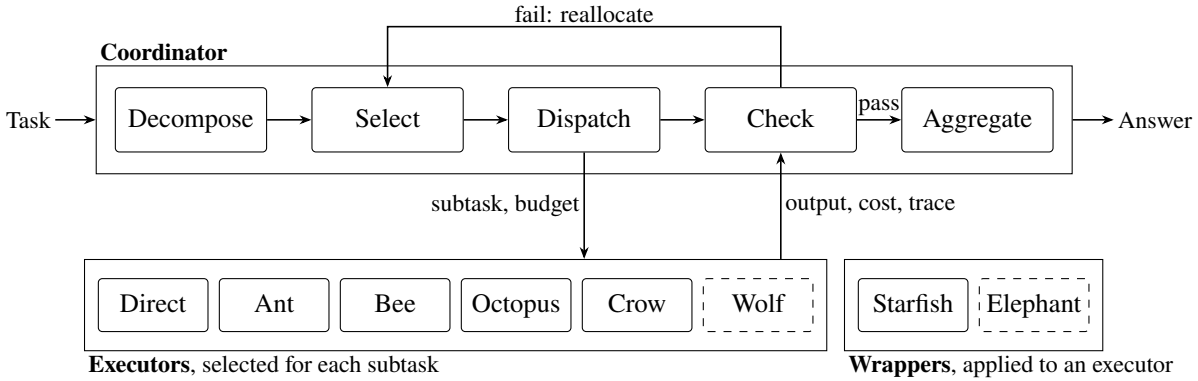


Figure 1. Planned architecture. Every outcome is logged and used to update the selector. Dashed modules are extensions.

4.2 Mechanisms

The modules are of two kinds (Table 3). An executor solves a subtask and is something the coordinator can select. A wrapper cannot solve a subtask by itself; it adds recovery or memory to whichever executor is running. Direct, ant, bee and octopus form the core pool and are built first. The starfish wrapper follows, since the fault study needs it. Crow is added afterwards as an extended pool, and wolf and elephant are attempted only if the core study is on schedule. The starfish module is named loosely: its model is the brittle-star robot of [28], and brittle stars and starfish are distinct groups.

Ant. The executor builds a solution as a sequence of choices, such as which tool to call or which sub-goal to try next. Choices come from a fixed vocabulary for the task family, so that what is learned on one path applies to the next. At each step one model call proposes and ranks the candidates, and the rank gives a prior η_c. A weight $\tau_{f,c}$ is kept for each task feature f and choice c, starting from τ_0, and a choice is sampled with probability proportional to $\tau_{f,c}^{\alpha}\,\eta_c^{\beta}$. When a finished path has been scored by the runtime checks with reward $r \in [0, 1]$, every weight under f evaporates and the choices on the path receive a deposit:

$$\tau_{f,c} \leftarrow \max\bigl(\tau_{\min},\ (1-\rho)\,\tau_{f,c}\bigr) \text{ for all } c, \qquad \tau_{f,c} \leftarrow \tau_{f,c} + \delta\, r \text{ for } c \text{ on the path.}$$

The three controls follow directly. With $\delta = 0$ nothing is reinforced and sampling follows the prior. With $\rho = 0$ nothing is forgotten. With $\alpha = \beta = 0$ the sampler is uniform.

Table 3. Modules as computational behavior. The last column lists the plain counterparts used as controls.

Module	Control flow	State kept	Stops when	Controls
Executors				
Direct	One model call, with tools if the subtask needs them	None	After the call	Reference point
Ant	Builds a solution as a sequence of choices, sampled by learned weight and model prior; repeats and scores each path	Weight for each task feature and choice	Best path is unchanged for several rounds, or budget is spent	No deposit; no evaporation; uniform sampler
Bee	Scouts answer independently, then recruit uncommitted scouts and send stop signals to supporters of rival options	Commitment of each scout, within one subtask	One option holds a quorum, or budget is spent	No stop signals; majority vote at equal cost
Octopus	Receives a goal and a budget, runs its own act–observe loop, returns a summary	Private scratchpad, dropped after the report	Goal check passes, or local budget is spent	Worker that reports to the coordinator after every step
Crow	Works back from the goal to the tool outputs it depends on; may write and cache a helper tool	Prerequisite graph and cached helper tools	Prerequisites are met and the final call passes its check	Step-by-step tool use without a plan
Wolf	Parallel explorers with a penalty on overlapping search	Shared record of covered ground	Coverage target or budget	Independent parallel explorers
Wrappers				
Starfish	Checkpoints the executor's messages, tool outputs and remaining budget; after a failure hands the remaining work to another executor	Checkpoints, within one task	Subtask completes, or retries are exhausted	Plain retry; restart of the subtask
Elephant	Retrieves similar past episodes with their outcomes and adds them to the executor's context	Episode store across tasks	After one retrieval per subtask	Same executor without retrieval

Bee. The module runs k scouts. Each starts uncommitted and answers the subtask independently, and scouts whose normalized answers match support the same option. In each later round an uncommitted scout is shown the argument of a committed scout, drawn in proportion to the current support of each option, and commits to that option if it accepts the argument. This is recruitment. A committed scout that receives a stop signal from a supporter of a rival option returns to the uncommitted state with probability σ. This is cross inhibition. The module answers as soon as one option is held by a quorum of θk scouts. If the budget runs out first it returns the option with the most support, and breaks ties by summed confidence. The procedure is our adaptation of the process described in [21] and not a fit of their model. Its controls are the same procedure with $\sigma = 0$, and a majority vote over the same number of model calls. All parameters of both modules are tuned on development tasks only.

4.3 Allocation policy

The arms of the selector are the executors, including the direct route. Wrappers are not arms. Starfish is switched on or off for a whole experiment, and elephant is evaluated as a component that is present or absent. The coordinator describes each ready subtask by features that are available before execution: the number of candidate options, whether a runtime check exists for the result, the tools it needs and the depth of their dependencies, the number of sibling subtasks that are ready at the same time, an estimate of its length, and the recorded outcomes of similar subtasks.

Allocation is developed in two stages. The first is a small rule table derived from the executor profiles of

RQ1. The second is a contextual bandit over the same features, LinUCB [34], as used for subtask routing in [15]. The reward of a choice is $s-\lambda c$, where c is the normalized token cost and λ sets the trade-off. The success term s has two sources. The runtime check of the subtask gives a signal that would be available in any deployment. On the training split each choice is also credited with the graded outcome of its parent task. That label exists for training only and is never shown to the coordinator on test tasks.

Two protocols keep learning and evaluation apart. In the *frozen* protocol the selector, the ant weights and the episode store are fixed after development, nothing is carried from one test task to the next, and all state is reset between seeds and between systems. It is used for RQ1, RQ2, RQ3 and RQ5. In the *online* protocol, used for RQ4, tasks arrive as an ordered stream with a controlled change in the task mix or the fault rate, and the selector keeps updating from runtime checks alone. It is compared with its own frozen copy on the same stream.

4.4 Evaluation plan

What the systems see. Reference answers stay inside the evaluation harness. They are used to grade the final answer of a task and, on the training split, to produce training labels. During a run a system sees only runtime checks: the status returned by a tool, the schema and type of an output, constraint checkers that are a public part of the task, tests supplied with the task, and agreement between independent samples. For a planning task the plan validator is a legitimate runtime check, because the domain model is part of the input. For a question-answering subtask no such check exists, and the executor has to rely on tool status, format and agreement. A subtask without a reliable check passes with its confidence estimate, and the selector learns from that weaker evidence. Every system, baselines included, gets the same checks and the same allowance of retries.

Tasks. Three families of 150 tasks each are planned. Planning uses Blocksworld and Logistics instances from PlanBench with its plan validator [35] and validation queries from TravelPlanner with its official evaluator [36]. Tool use uses GAIA validation questions of levels 1 and 2 that can be answered with text and file tools [37], graded by normalized exact match, together with a small suite of our own built on deterministic local tools. Reasoning uses GSM8K and MATH, as in [1]. Our models and subsets differ from the published ones, so the numbers will allow only a rough comparison. We will also generate synthetic tasks in which the number of alternatives, the dependency depth, the parallel width and the fault rate are set by us, so that the hypotheses of RQ1 can be tested one factor at a time. The 450 tasks are split at the level of the task or task template, before any subtasks exist, into training (150), tuning (90) and held-out test (210) sets with equal shares of each family. One task variant per family appears only in the test set.

Systems compared. (i) A single agent with the same tools and retries. (ii) Each core executor used alone. The best of them is chosen on the tuning split; the best in hindsight on the test set is reported only as an upper bound. (iii) Fixed allocation, with one executor assigned to each task family by hand. (iv) Random allocation. (v) A conventional coordinator with specialist agents: a planner that reads the task graph, an executor role that works each subtask in an act–observe loop, and a critic that reviews each result and may ask for one revision. It has the same decomposer, checks, tools, token cap and retries, and its prompts are tuned on the tuning split with the same number of tuning runs as ours. (vi) A conventional adaptive system: the same LinUCB selector choosing among ordinary workers (direct call, act–observe loop, majority vote, plan-then-execute). This baseline has adaptivity without the biological mechanisms and is the direct test of RQ3. (vii) Adaptive allocation over the core pool, rule-based and learned. Where its released code runs on our backbone, MasRouter [1] will be added on the reasoning family as an outside reference; it is not part of the core comparison.

Primary comparison and statistics. One comparison is fixed in advance as primary: learned adaptive allocation against the best single executor chosen on the tuning split, on the held-out test set, at a token cap of twice the mean cost of the single agent, with task success as the measure. The cost claim uses a non-inferiority margin of five percentage points, so lower cost counts only if success is not worse by more

than that. Each configuration is run with three seeds, and each run is a fresh sample. Repeated runs of one task are not independent tasks, so uncertainty is estimated with a paired bootstrap that resamples tasks and averages over seeds within a task. All other comparisons (the remaining baselines, single families, other budgets, ablations) are secondary, are reported with Holm correction and are labeled exploratory. When an interval includes zero we will report that no difference was detected and will not read it as equivalence.

Measurements. Task success; tokens divided into coordinator, worker and inter-agent communication; model calls; elapsed time on uncached runs; and successes per million tokens. Success is also reported at half and at twice the primary token cap for four systems, since a single operating point can favor any method. Routing regret is measured on a replay set of about 200 subtask contexts saved from development runs, each executed by every core executor from the same state; the regret of a choice is the reward of the best executor on that context minus the reward of the chosen one.

Faults and recovery. Three faults are injected at fixed rates: a tool call that fails once, a worker that times out for the rest of the task, and a corrupted intermediate result. The third follows the error injection of [29]; the first two are our additions. The recovery rate is the share of fault-affected tasks that still end with a correct answer. Starfish is compared with plain retry and with restarting the subtask. A sample of failed runs will be labeled with the taxonomy of [3].

Threats to validity. Hosted models change without notice, so model versions will be pinned and the primary comparison repeated on an open-weight model with fixed weights. A response cache is used for debugging and exact replay only. It is switched off for measured runs, and latency is taken from uncached runs. Public benchmarks may have been seen in training, which is one reason for the synthetic and in-house tasks. With 210 test tasks the study can detect only fairly large effects, roughly eight percentage points or more in success, and the report will state this limit next to every claim.

5 Technical Requirements

Hardware	Two personal laptops for development. The reference machine has an Intel Core i5-1240P, 16 GB RAM and an NVIDIA RTX 2050 (4 GB) and runs Linux; this is enough for the orchestration code, the tool sandbox and small local models for smoke tests. Free cloud notebooks (Kaggle, Google Colab) with a 16 GB T4-class GPU for the open-weight model. About 20 GB of storage for traces. No dedicated server or departmental GPU is required.
Software	Python 3.11 or later with `asyncio`; Pydantic for message schemas; NetworkX for the task graph; SQLite and JSON Lines for traces; NumPy and scikit-learn for the selector; pandas, SciPy and Matplotlib for analysis; pytest for tests; Docker for sandboxed tool execution; Hugging Face Transformers and Datasets, with Ollama or `llama.cpp` to serve the open-weight model; Git and GitHub for version control (`https://github.com/Nikshay1/BTP`); LaTeX for the report. The orchestration layer will be written by us instead of being taken from an agent framework, because every message has to be metered.
Models	One small hosted model as the primary backbone, reached through an API with a free or low-cost tier (for example the Gemini API, Groq or OpenRouter), and one open-weight instruction-tuned model of 7–8 billion parameters (Llama, Qwen or Mistral family) in 4-bit quantization as the second backbone. The exact models will be fixed after the pilot and then kept unchanged.

Compute. No model is trained; the cost is inference. Table 4 lists every experiment in the plan and comes to about 20,700 task runs on the hosted model. At 8,000 to 15,000 tokens per run this is roughly 165 to 310 million tokens. As a planning figure we assume four fifths of the tokens are input and a list price of US$0.10 per million input tokens and US$0.40 per million output tokens, which is the level of small hosted models at the time of writing. That gives US$26 to US$50, or up to about US$65 with a 30% allowance for retries, failed runs and prompt iteration (roughly Rs. 2,000 to 5,500). The price,

the token split and the tokens per run will be measured in the pilot and the estimate revised. The open-weight replication is about 1,260 task runs. At the 20 to 30 generated tokens per second we expect from a quantized 8B model on a T4 it needs some 40 to 60 GPU-hours, spread over several weeks of free quota. Free GPU time is not guaranteed, so this replication is the first item to shrink. If money or quota falls short, the minimum study is the profiling, the main comparison and the executor ablations, about 11,800 task runs.

Table 4. Experiment matrix and planning estimate of task runs on the hosted model.

Experiment	**Configurations × tasks × seeds**	**Task runs**
Executor profiles on training and tuning tasks (RQ1)	$5 \times 240 \times 3$	3,600
Tuning of selectors and baselines	$4 \times 90 \times 3$	1,080
Main comparison on test tasks (RQ2, RQ3)	$10 \times 210 \times 3$	6,300
Executor ablations (RQ3)	$6 \times 105 \times 3$	1,890
Two further budget levels (RQ5)	$4 \times 105 \times 3 \times 2$	2,520
Fault study, three recovery variants, three faults (RQ4)	$9 \times 105 \times 3$	2,835
Online stream, updating against frozen (RQ4)	$2 \times 210 \times 3$	1,260
Replay set for routing regret	4×200 subtasks $\times 3$	600
Extended pool with crow	$1 \times 210 \times 3$	630
Total		20,715

The ten configurations of the main comparison are the single agent, three core executors alone, fixed, random, specialist, conventional adaptive, and adaptive allocation in two forms. The replay set is 2,400 subtask runs, counted at four subtasks per task.

6 Expected Outcomes

- **A modular research prototype.** A coordinator with a common interface for heterogeneous bio-inspired agents, validated first with three mechanisms and able to take all seven.
- **Capability profiles.** Evidence of which mechanism suits which class of subtask, and of when combining them is useful.
- **Comparative results.** Task success, token use, inference cost, completion time and recovery after controlled failures, for the adaptive system and for every baseline at equal budget.
- **Component-level analysis.** Ablations that separate the effects of adaptive routing, memory, communication and the agent-specific mechanisms.
- **Reproducible materials.** Code, configurations, evaluation scripts and experiment logs in a public repository, supporting the final report and a possible conference submission.

The working hypothesis is that adaptive allocation improves task success at an equal budget, or lowers cost at comparable success. It is tested by the primary comparison of Section 4.4. If the conventional adaptive system or the specialist coordinator matches the bio-inspired system within the uncertainty of the experiment, the report will state that no additional benefit of the biological mechanisms was detected. Cases in which a single agent is the better choice are the answer to RQ5. As in the synopsis, the project maps to SDG 9 (target 9.5) as primary and SDG 8 (target 8.2) as secondary, and no energy or carbon claim will be drawn from token savings.

7 Timeline and Work Plan

The plan covers the seventh and eighth semesters, from October 2026 to May 2027. Each column of Figure 2 is half a month. Evaluation dates are indicative and will follow the department schedule.

The seventh semester is for building. By its end the core executors, the task suites with their splits and the baselines should run end to end, and the pilot should tell us what a task run costs and whether the mechanisms differ at all. The eighth semester is for the study itself. The core pool is frozen before the selector is trained, so that the main comparison is made on a fixed set of executors, and crow is evaluated afterwards with a retrained selector and reported separately. The main comparison starts only when the pilot has confirmed the cost of a run and the runtime checks of every task family have passed on a hand-labeled sample. The bars that cross the end-semester examinations are deliberately long, since little project work will be done in those weeks.

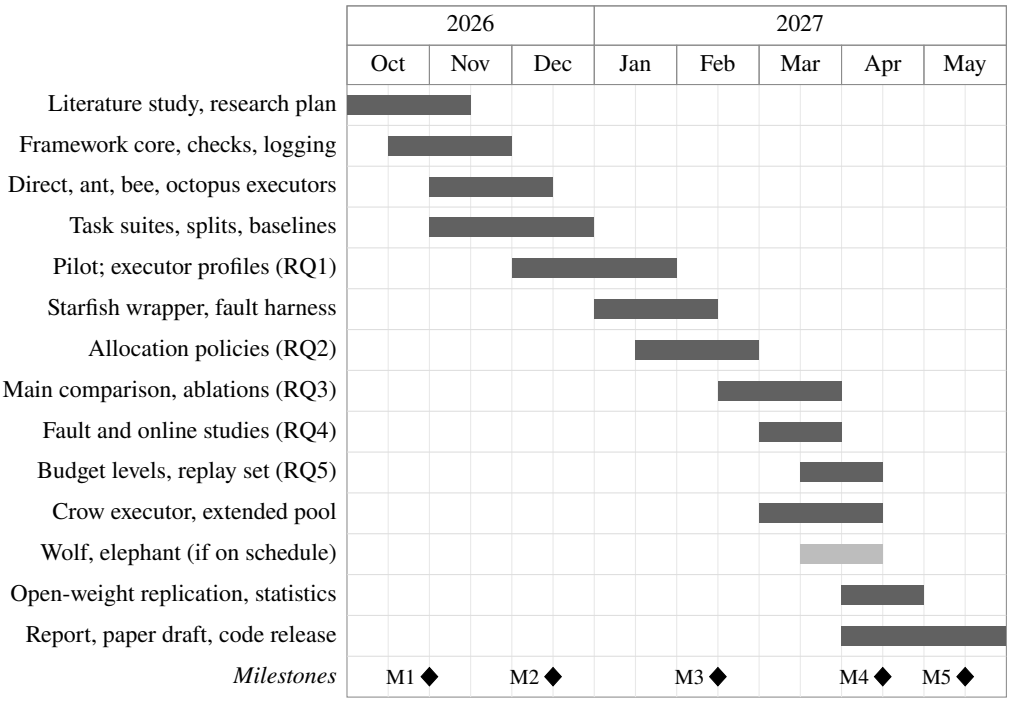


Figure 2. Work plan, October 2026 to May 2027. The lighter bar is optional work.

Table 5. Milestones and deliverables.

No.	When	Deliverable
M1	Mid-November 2026	Coordinator, common interface, runtime checks and logging working end to end with one executor on a toy task.
M2	December 2026	Core executors, task suites with fixed splits, and baselines; pilot results with the measured cost of a task run, for the seventh-semester evaluation.
M3	End of February 2027	Executor profiles (RQ1); core pool frozen; both allocation policies trained and tuned (RQ2); starfish wrapper ready.
M4	End of April 2027	Main comparison and ablations on held-out tasks (RQ2, RQ3), fault and online studies (RQ4), budget levels (RQ5) and the open-weight replication.
M5	May 2027	Final report, public code and logs, and a paper draft for the final evaluation.

Table 6. Main risks and how the plan handles them.

Risk	Response
API limits or cost exceed the estimate	Develop on small subsets with small models, and fall back to the minimum study of Section 5.
Free GPU quota is not available	The open-weight replication is reduced or dropped, and the report says so.
Results vary between runs or after a model update	Pinned model versions, three fresh seeds for each configuration and a paired analysis over tasks.
The mechanisms do not differ in strengths	This is an answer to RQ1. The study then concentrates on cost and overhead (RQ5) and the selector is reduced to choosing between direct and coordinated execution.
Seven modules are too many for two semesters	The main study uses the core pool only. Crow is evaluated afterwards as an extended pool; wolf and elephant only if the core study is on schedule.
Runtime checks are unreliable for a task family	Checks are validated on a hand-labeled sample before the main comparison; a family that fails is dropped from it.

References

[1] Y. Yue, G. Zhang, B. Liu, G. Wan, K. Wang, D. Cheng, and Y. Qi, "MasRouter: Learning to route LLMs for multi-agent systems," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2025, pp. 15549–15572.

[2] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "GPTSwarm: Language agents as optimizable graphs," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 235, 2024, pp. 62743–62767.

[3] M. Cemri *et al.*, "Why do multi-agent LLM systems fail?," in *Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, 2025.

[4] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision, and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, 2025, doi: 10.1145/3712003.

[5] X. Wu, S.-H. Wu, J. Wu, L. Feng, and K. C. Tan, "Evolutionary computation in the era of large language model: Survey and roadmap," *IEEE Trans. Evol. Comput.*, vol. 29, no. 2, pp. 534–554, 2025, doi: 10.1109/TEVC.2024.3506731.

[6] Y. Kim *et al.*, "Towards a science of scaling agent systems," arXiv:2512.08296, version 3, 2026.

[7] Q. Wang, Z. Wang, Y. Su, H. Tong, and Y. Song, "Rethinking the bounds of LLM reasoning: Are multi-agent discussions the key?," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2024, pp. 6106–6131.

[8] G. Zhang *et al.*, "Cut the crap: An economical communication pipeline for LLM-based multi-agent systems," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[9] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, "A dynamic LLM-powered agent network for task-oriented agent collaboration," in *Proc. 1st Conf. Lang. Model. (COLM)*, 2024.

[10] G. Zhang *et al.*, "G-Designer: Architecting multi-agent communication topologies via graph neural networks," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, 2025.

[11] J. Zhang *et al.*, "AFlow: Automating agentic workflow generation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[12] I. Ong *et al.*, "RouteLLM: Learning to route LLMs with preference data," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[13] T. Feng, H. Zhang, Z. Lei, P. Han, and J. You, "GraphPlanner: Graph memory-augmented agentic routing for multi-agent LLMs," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.

[14] G. Yu, "AdaptOrch: Task-adaptive multi-agent orchestration in the era of LLM performance convergence," arXiv:2602.16873, 2026.

[15] Z. Guan *et al.*, "Symphony-Coord: Adaptive routing for multi-agent LLM systems," arXiv:2602.00966, 2026.

[16] X. Wang *et al.*, "Efficient and interpretable multi-agent LLM routing via ant colony optimization," arXiv:2603.12933, 2026.

[17] R. Li *et al.*, "SwarmSys: Decentralized swarm-inspired agents for scalable and adaptive reasoning," arXiv:2510.10047, 2025.

[18] M. A. U. Rahman, M. Schranz, and S. Hayat, "LLM-powered swarms: A new frontier or a conceptual stretch?," *IEEE Intell. Syst.*, early access, 2026, doi: 10.1109/MIS.2026.3728883.

[19] R. Damaševičius and R. Maskeliūnas, "Agent state flipping based hybridization of heuristic optimization algorithms: A case of bat algorithm and krill herd hybrid algorithm," *Algorithms*, vol. 14, no. 12, art. 358, 2021.

[20] C. L. Camacho-Villalón, M. Dorigo, and T. Stützle, "Exposing the grey wolf, moth-flame, whale, firefly, bat, and antlion algorithms: Six misleading optimization techniques inspired by bestial metaphors," *Int. Trans. Oper. Res.*, vol. 30, no. 6, pp. 2945–2971, 2023.

[21] T. D. Seeley, P. K. Visscher, T. Schlegel, P. M. Hogan, N. R. Franks, and J. A. R. Marshall, "Stop signals provide cross inhibition in collective decision-making by honeybee swarms," *Science*, vol. 335, no. 6064, pp. 108–111, 2012.

[22] X. Wang *et al.*, "Self-consistency improves chain of thought reasoning in language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[23] G. Sumbre, Y. Gutfreund, G. Fiorito, T. Flash, and B. Hochner, "Control of octopus arm extension by a peripheral motor program," *Science*, vol. 293, no. 5536, pp. 1845–1848, 2001.

[24] A. H. Taylor, G. R. Hunt, J. C. Holzhaider, and R. D. Gray, "Spontaneous metatool use by New Caledonian crows," *Curr. Biol.*, vol. 17, no. 17, pp. 1504–1507, 2007.

[25] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, "Large language models as tool makers," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[26] S. Kim *et al.*, "An LLM compiler for parallel function calling," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, 2024.

[27] Y. Qin *et al.*, "Tool learning with foundation models," *ACM Comput. Surv.*, vol. 57, no. 4, pp. 1–40, 2025, doi: 10.1145/3704435.

[28] T. Kano, E. Sato, T. Ono, H. Aonuma, Y. Matsuzaka, and A. Ishiguro, "A brittle star-like robot capable of immediately adapting to unexpected physical damage," *R. Soc. Open Sci.*, vol. 4, no. 12, art. 171200, 2017.

[29] J.-T. Huang *et al.*, "On the resilience of LLM-based multi-agent collaboration with faulty agents," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, 2025.

[30] S. Zhang *et al.*, "Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems," in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, 2025.

[31] G. Theraulaz, E. Bonabeau, and J.-L. Deneubourg, "Response threshold reinforcements and division of labour in insect societies," *Proc. R. Soc. Lond. B*, vol. 265, no. 1393, pp. 327–332, 1998.

[32] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 17, 2024, pp. 19632–19642.

[33] Z. Zhang *et al.*, "A survey on the memory mechanism of large language model-based agents," *ACM Trans. Inf. Syst.*, vol. 43, no. 6, pp. 1–47, 2025, doi: 10.1145/3748302.

[34] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in *Proc. 19th Int. Conf. World Wide Web (WWW)*, 2010, pp. 661–670.

[35] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati, "PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change," in *Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, 2023.

[36] J. Xie *et al.*, "TravelPlanner: A benchmark for real-world planning with language agents," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, 2024.

[37] G. Mialon, C. Fourrier, C. Swift, T. Wolf, Y. LeCun, and T. Scialom, "GAIA: A benchmark for general AI assistants," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.